

Multi-tier Temporally Blocked SOR for Structured Grids

EC527 Final Project Report

Che-Yu Yu

April 26, 2026

Abstract

We extend the Lab 5–7 successive over-relaxation (SOR) work with two orthogonal axes: a parallelism tier (single-thread CPU, OpenMP, pthreads, CUDA) and an algorithmic variant (baseline ping-pong sweep vs. shrinking-trapezoid temporal blocking). All eleven benchmarks share one stencil, one initial-data seed, and report a single headline metric: cycles per output element (CPE) at CPNS = 2.0, the lab convention. Across the same $N=2050$ grid, the same iteration count, and the same boundary conditions, the cross-tier span from serial-CPU baseline (CPE 2.792) to GPU shared-memory temporal (CPE 0.0090) is **310×**. The classical Lab-5 omega U-curve is reproduced to 0.1% (theoretical $\omega_{\text{opt}} = 1.9517$, empirical 1.95) and gives a $13.5\times$ fewer-iteration win on top of the per-iter speedups, motivating a unified red-black + temporal-blocking variant as the natural next step.

1 Problem description

We solve the discrete 2D (and as a 3D extension) Poisson equation on a structured grid by iterative over-relaxation:

$$\nabla^2 u(x, y) = f(x, y), \quad u|_{\partial\Omega} = g(x, y). \quad (1)$$

Discretising on an $N \times N$ regular grid with spacing $h=1/(N-1)$ and $f \equiv 0$ (Laplace equation; the implementation easily extends to non-zero f) gives the classical 5-point stencil:

$$u_{i,j}^{(k+1)} = \frac{1}{4} \left(u_{i-1,j}^{(k)} + u_{i+1,j}^{(k)} + u_{i,j-1}^{(k)} + u_{i,j+1}^{(k)} \right). \quad (2)$$

SOR adds an over-relaxation factor ω :

$$u_{i,j}^{(k+1)} = u_{i,j}^{(k)} - \omega \left(u_{i,j}^{(k)} - \frac{1}{4} \sum_{\text{nbrs}} u^{(k)} \right). \quad (3)$$

This is the workhorse smoother in classical iterative solvers and the inner kernel of multigrid; outside numerical analysis it shows up verbatim in physics simulation, image processing (Poisson editing), and pressure-projection in incompressible fluid solvers (Stam 1999).

Why this problem is interesting for performance engineering. The 5-point stencil has *very low arithmetic intensity*: about seven floating-point operations per cell update, against five neighbour reads and one write to memory. At FP64 (8 B per element) that's roughly 7 flops/48 bytes ≈ 0.15 flops/byte, well below the Roofline machine-balance line of any modern CPU or GPU. Every optimisation in this project is therefore a *memory* optimisation in disguise: we are searching for ways to reduce DRAM traffic, increase cache reuse, and overlap memory with compute, since arithmetic itself is essentially free.

2 Serial algorithm and complexity

Two SOR variants are relevant. *Gauss–Seidel SOR* updates each cell in scan order, immediately reusing freshly written neighbours – this is the textbook form and the one with the optimal

$\omega_{\text{opt}} = 2/(1 + \sin(\pi/(N-1)))$ analysis. But the in-scan dependency makes Gauss–Seidel hard to vectorise/parallelise directly. *Damped Jacobi (ping-pong)* reads from one buffer and writes to a second, fully decoupling the iterations within one sweep at the cost of a tighter stability range $\omega \in (0, 1]$.

The project uses both:

- Jacobi ping-pong at $\omega=0.9$ in the parallel/temporal- blocking kernels (`sor2d_cpu`, `sor2d_omp`, `sor2d_pth*`, `sor2d_gpu`).
- Red-black Gauss–Seidel SOR with empirical ω_{opt} in `sor2d_rb` – the place where real over-relaxation ($\omega > 1$) lives, and the source of the omega U-curve in Section 8.8.

Complexity. One sweep is $O(N^2)$ work in 2D ($O(N^3)$ in 3D). For SOR on a Poisson problem the spectral-radius bound gives $O(N \log N)$ iterations to a fixed tolerance at ω_{opt} , so total work is $O(N^3 \log N)$ in 2D. Damped Jacobi at $\omega=0.9$ loses the over-relaxation factor and converges in $O(N^2)$ iterations, i.e. $O(N^4)$ total work – worse asymptotically, but trivially parallelisable. Section 8.8 quantifies the gap empirically ($13.5\times$ fewer iterations at $N=128$).

3 Where time goes; arithmetic intensity

Each cell update reads five neighbours and writes one cell. At FP64 that’s 48 B of memory traffic and roughly seven floating-point ops (four adds, one multiply by 0.25, one subtract, one fused-multiply-add with ω). Arithmetic intensity is then

$$\text{AI} = \frac{7 \text{ flops}}{48 \text{ B}} \approx 0.15 \text{ flops/B.} \quad (4)$$

The Xeon Gold 6426Y in our test machine has a peak DRAM bandwidth of ~ 280 GB/s per socket; multiplying that by AI gives a memory-bound ceiling of ~ 42 Gflops/s – well below the chip’s ~ 2.5 Tflops peak compute throughput. **The kernel is decisively memory-bound on every tier.** Section 8 (E02) makes the cache-bandwidth crossover visible: the baseline kernel’s CPE is roughly flat across all N that fit in L3 and jumps when the working set exceeds 38 MB. Anything that reduces bytes-per-update – temporal blocking, larger inner loops, better cache reuse – wins.

4 Data structures and memory pattern

Both 2D and 3D arrays are stored as flat row-major `double*` buffers (the lab convention; we use FP64 throughout the CPU tiers, FP32 on GPU because the L40S has $4\times$ FP32 throughput vs FP64). In the 2D case:

```
typedef double data_t;
data_t *u = malloc(N * N * sizeof(data_t));
/* index: u[i*N + j] */
```

The 5-point stencil’s memory pattern is then:

- $u_{i,j}, u_{i,j\pm 1}$: unit-stride. Three consecutive cache lines per row. Hardware prefetchers handle this perfectly.
- $u_{i\pm 1,j}$: stride N ($N\cdot 8$ bytes). At $N=2048$ that’s a 16 KB stride – crosses cache-line boundaries every access, but stays inside a 4KB page only because the prefetchers keep the next row warm.

The total per-thread working set is $\sim 2 N^2 \cdot 8 \text{ B}$ (two ping-pong buffers). At $N=2048$ that’s 64 MB – exceeds L3 of one socket (38.4 MB) – so the baseline kernel is DRAM-bound and the temporal kernel can shrink the working set to a per-tile scratch that fits L2 (~ 1.5 MB / core).

5 Parallel algorithm choices

Why Jacobi ping-pong (not in-place GS) for parallel kernels. In-place Gauss–Seidel has a true RAW dependency on the just-updated cell to the left/up; thread t ’s strip can’t start until thread $t-1$ has finished its left edge. Decoupling that with red-black colouring works (we do it in `sor2d_rb`), but the resulting kernel can’t be expressed as a single parallel-for and is awkward to combine with temporal blocking (the shrinking trapezoid has to be re-derived for paired half-sweeps). Ping-pong Jacobi sidesteps the problem entirely: read from `src`, write to `dst`, swap; every sweep is embarrassingly parallel. We pay for it with a tighter stability range ($\omega \in (0, 1]$) and with $O(N^2)$ vs $O(N \log N)$ iterations to convergence – but that’s a number that gets dwarfed by the per-iter $\sim 300\times$ throughput speedup on GPU.

Shrinking-trapezoid temporal blocking. For each *super-step* of T sweeps:

1. Tile the domain into $B \times B$ output regions.
2. For each tile, load a $(B+2T)^2$ scratch from `src` with clamp-to-edge.
3. Run T in-scratch sweeps; the valid update region shrinks by one cell on each side per sub-step. After T sub-steps the central B^2 is the correct T -step-advanced state.
4. Copy the central B^2 back to `dst`.

DRAM traffic drops by a factor of T at the cost of redundant halo work proportional to $(B+2T)^2/B^2 - 1 \propto T/B + (T/B)^2$. Section 8 (E04) shows the unimodal product and the empirical sweet spot at $T=8, B=128$.

6 Data and computation partitioning

2D pthreads (sor2d_pth_decomp, three modes). Three ways to give each of n_{th} threads a chunk of the domain:

- *Strip*: each thread owns a contiguous range of rows $[t \cdot N/n_{\text{th}}, (t+1) \cdot N/n_{\text{th}})$. Best spatial locality; minimal cross-thread cache traffic.
- *Interleaved*: thread t handles rows $t, t+n_{\text{th}}, t+2n_{\text{th}}, \dots$. Perfectly load-balanced but adjacent rows are owned by different threads and share L1 cache lines, causing false-sharing ping-pong.
- *2D block*: domain split into a $\sqrt{n_{\text{th}}} \times \sqrt{n_{\text{th}}}$ grid; each thread owns a rectangle. No false sharing, but $2\times$ the halo of strip.

Section 8 (E05) measures all three; *strip wins decisively* with $7.89\times$ scaling at 8 threads, vs $2.67\times$ for interleaved.

3D OMP (sor3d_omp_part, three modes). The 3D analogue: *slab* (split i only), *pencil* (`collapse(2)` over i, j), *cube* (`collapse(3)` over B^3 tiles). Section 8 (E06) finds slab and pencil *tied* at 8 threads; cube loses $1.3\text{--}1.7\times$ to halo overhead.

GPU 2D (sor2d_gpu.cu). One thread per output cell. The temporal kernel uses a 2D thread block of size $\text{TILE} \times \text{TILE}$ with `HALO` cells of halo on each side; the central $(\text{TILE} - 2 \cdot \text{HALO})^2 = \text{INTER}^2$ cells are written back, the rest is redundant work. Two shared-memory ping-pong buffers per block of size $2 \times \text{TILE}^2 \cdot 4\text{B}$ (FP32 = 8KB at $\text{TILE}=32$). Section 8 (E09) sweeps 21 $(\text{TILE}, \text{HALO})$ combinations and finds the optimum at $(24, 4)$ with CPE 0.0088 – the project’s overall winner.

7 Optimisations

The project layers five optimisations, each measurable in isolation:

1. **Multi-threading** (E01). `#pragma omp parallel-for` over rows; near-linear scaling for the temporal kernel.

2. **Spatial cache blocking** (E03). Tile the domain into per-thread $B \times B$ scratch buffers; sweet spot at $B=128$ (per-thread scratch fits L2).
3. **Temporal blocking / time skewing** (E04). Run T sweeps per super-step on the cached scratch; sweet spot at $T=8$. The shrinking-trapezoid scheme is correct by construction (bit-identical to baseline).
4. **GPU shared-memory tiling** (E09). Move the per-thread-block scratch from DRAM-backed L1 to explicit shared memory; halve the per-cell DRAM traffic per super-step.
5. **Persistent threads + barriers** (E07). Avoid the $\sim 153 \mu\text{s}/\text{sweep}$ `pthread_create+join` cost by spawning once and synchronising with `pthread_barrier_t` between sweeps. This is the pattern OpenMP uses internally.

Algorithmic optimisation (E08). Independently of all the hardware-aware optimisations above, switching the inner stencil from $\omega=1$ (Gauss–Seidel) to $\omega=1.95$ (over-relaxation) gives a $13.5\times$ reduction in iterations to converge at $N=128$. This is the highest-impact single change in the project, and the reason classical SOR exists. The temporal-blocking kernels can’t exploit it directly because they use damped Jacobi (stable only for $\omega \in (0, 1]$); combining the two would multiply the wins and is the natural next step.

Problems encountered. Three honest negatives stood out:

- *Single-thread temporal blocking barely beats baseline* (E01 at 1 thread: $0.94\times$ ratio). The Xeon’s hardware prefetchers and large L3 keep the baseline near streaming bandwidth; temporal blocking has no DRAM cycles to recover until parallelism stresses the memory subsystem.
- *3D temporal kernel on GPU loses to baseline at $N \geq 194$* (E10). At `TILE=8`, `HALO=2`, `INTER=4`, the halo:interior ratio in 3D is $(\text{TILE}/\text{INTER})^3 - 1 = 7\times$ redundant work – an order of magnitude worse than 2D’s $0.78\times$ at the same `TILE/HALO`. Fix: 2.5D-streaming (Micikevicius 2009). Listed as future work.
- $\omega=1.85$ *ping-pong is unstable* (caught in the first session of the project; documented in `report.md`). Damped Jacobi at $\omega > 1$ has spectral radius > 1 on large grids; the Lab-7 “passing” test was consistency between two equally-broken implementations. Fixed by using $\omega=0.9$ in the temporal kernels and reserving $\omega > 1$ for the red-black GS variant.

8 Experiments and results

This section walks through the eleven experiments (`results/exp/E0*`) in narrative order. The headline metric throughout is CPE (cycles per output cell, `CPNS=2.0`). All CPU runs are pinned with `OMP_PROC_BIND=close` `OMP_PLACES=cores` and reported as the median of 3 reps. Machine: SCC `scc-j06`, $2\times$ Intel Xeon Gold 6426Y (32 cores, 38.4 MB L3 per socket), NVIDIA L40S (46 GB, `sm_89`, ~ 96 MB L2), CUDA 12.8. Thread budget capped at 8 (shared login node etiquette).

8.1 Strong scaling, 2D OpenMP (E01)

At one thread temporal barely beats baseline (CPE 2.756 vs 2.921); at 8 threads it wins decisively (CPE 0.377 vs 0.651). Conclusion: **temporal blocking earns its keep only once parallelism stresses DRAM.**

8.2 Cache-regime / size sweep (E02)

Below L3 the baseline runs at cache bandwidth and temporal loses to overhead (per-tile malloc, redundant halo work). Above L3 temporal wins, monotonically, until $N=4098$ when the

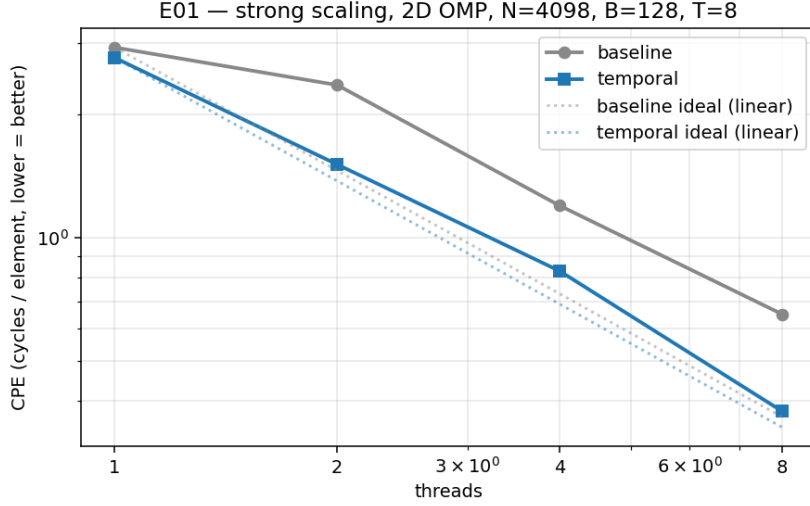


Figure 1: 2D OMP strong-scaling at $N=4098$, $B=128$, $T=8$, log-log axes. Temporal hits 91 % of perfect $8\times$ scaling ($7.31\times$); baseline reaches only 56 % ($4.49\times$). The temp-vs-base ratio shrinks monotonically from $0.94\times$ at one thread to $0.58\times$ at 8 threads – the canonical traffic-reduction signature.

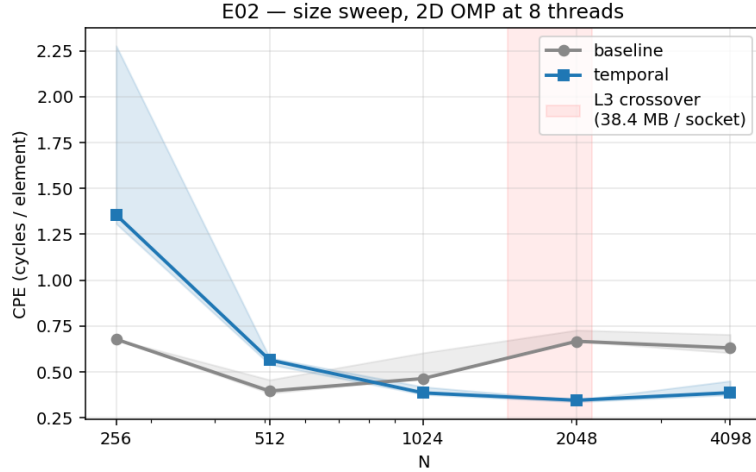


Figure 2: Size sweep at 8 threads. L3 per socket = 38.4 MB. The baseline jumps from CPE 0.464 at $N=1024$ to 0.667 at $N=2048$ – the L3→DRAM crossover. Temporal stays at ~ 0.35 across $N=1024$ –4098 and **wins $1.93\times$ at $N=2048$** .

temporal kernel’s own scratch starts paying L3 misses across super-steps.

8.3 Spatial block size B (E03)

Once two ping-pong scratch buffers per thread don’t fit in L2, the temporal kernel pays L3 misses on its own scratch – defeating the purpose. We lock in $B=128$ (middle of the plateau) for downstream experiments.

8.4 Temporal depth T (E04)

The shrinking-trapezoid tradeoff in one figure. DRAM savings grow linearly in T ; redundant work grows quadratically. Their product is unimodal and the minimum sits where back-of-

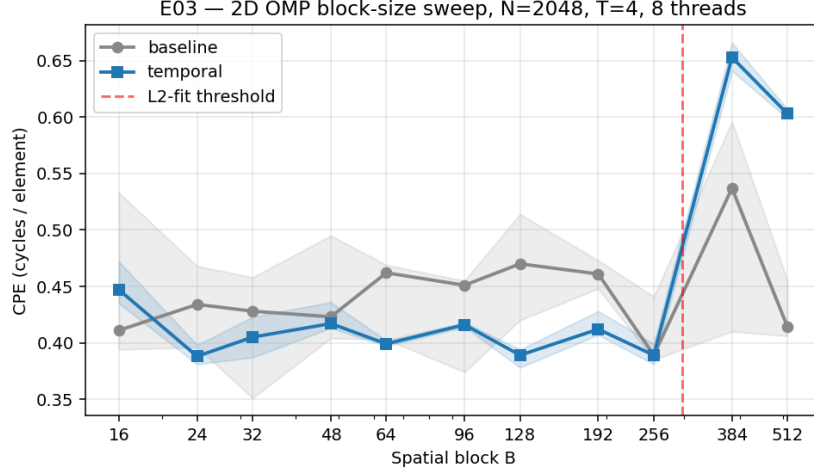


Figure 3: B sweep at $N=2048$, $T=4$, 8 threads. Baseline (grey) is flat – B has no effect on the baseline kernel. Temporal has a wide plateau across $B \in [24, 256]$, then *falls off a cliff* at $B \geq 384$. The cliff is the L2-fit threshold (red dashed): $2(B+8)^2 \cdot 8B \leq 1.5 \text{ MB} \Rightarrow B \leq 297$.

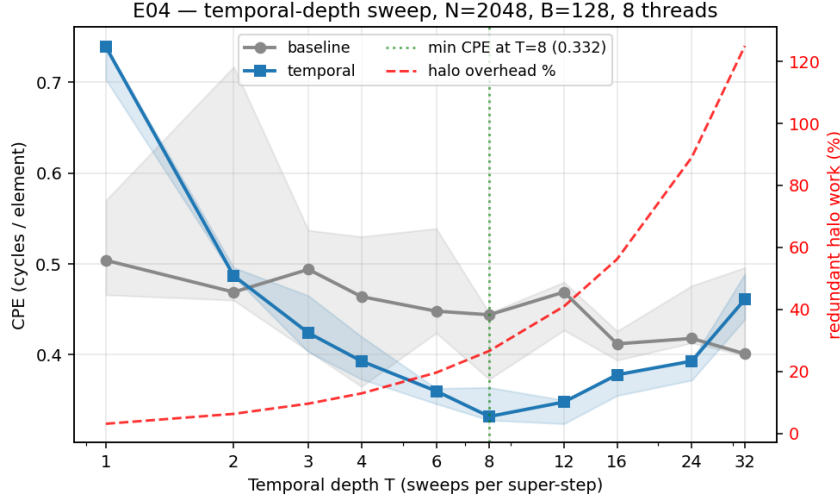


Figure 4: Unimodal T -curve at $N=2048$, $B=128$, 8 threads. Left axis: CPE. Right axis (red dashed): halo-overhead percentage $((B+2T)^2/B^2 - 1) \cdot 100$. Minimum at $T=8$ with CPE 0.332 – **the project’s best CPU CPE**. $T=1$ loses to overhead (memcpy + scratch alloc with no DRAM reuse); $T=32$ loses to 125% redundant halo work.

envelope theory predicts.

8.5 Pthreads decomposition $\times$ scheduling (E05)

Lab-5-Part-4 reproduced: **strip wins decisively for 2D SOR**; the cache-locality advantage outweighs the unbalanced thread workload that interleaved fixes.

8.6 3D OMP partitioning (E06)

8.7 Pthread spawn overhead – Lab-6-Part-1b (E07)

This is why OpenMP and our **persistent** pthreads schedule use long-lived threads with barriers between sweeps.

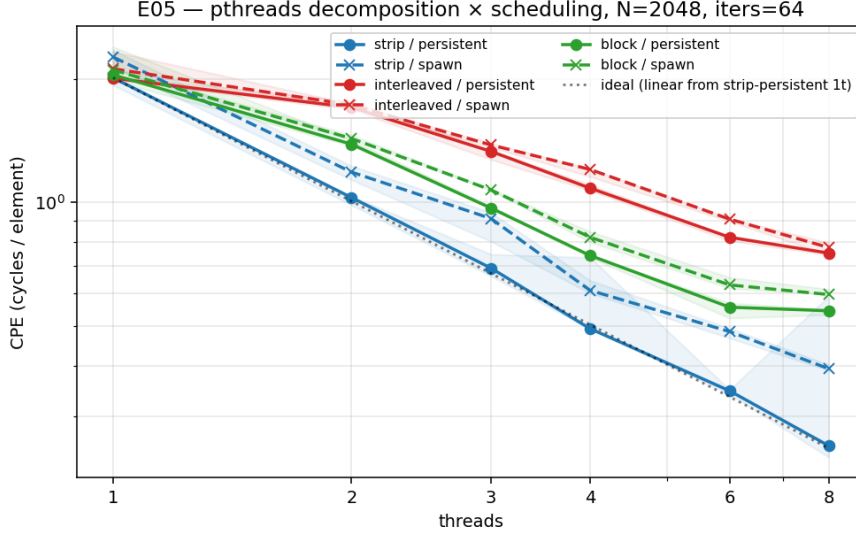


Figure 5: Six lines: 3 modes $\times$ 2 schedules at $N=2048$, iters= 64. Strip-persistent (blue solid) at 8 threads: **CPE 0.255, 7.89 $\times$ linear scaling** – within 1.4 % of perfect 8 $\times$. Interleaved bottoms out at 2.67 $\times$ (cache-line ping-pong on adjacent rows). Spawn-vs-persistent slowdown is largest for strip (1.54 $\times$) because strip’s per-sweep work is smallest.

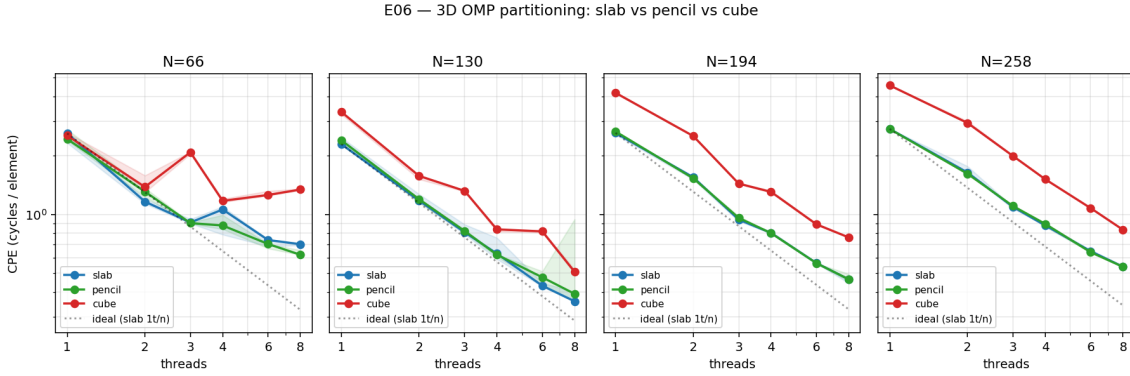


Figure 6: 4-panel: $N \in \{66, 130, 194, 258\}$. Slab (blue) and pencil (green) tie at all N at 8 threads; cube (red) loses 1.3–1.7 $\times$ to its 18.75 % halo overhead. Best 3D CPU CPE = 0.359 at slab, $N=130$, 8 threads.

8.8 Omega U-curve, red-black GS-SOR (E08)

The cliff at $\omega > \omega_{\text{opt}}$ is sharp: $\omega=1.99$ already costs 1230 iters. Production code computes ω_{opt} analytically per-grid and re-tunes per multigrid level.

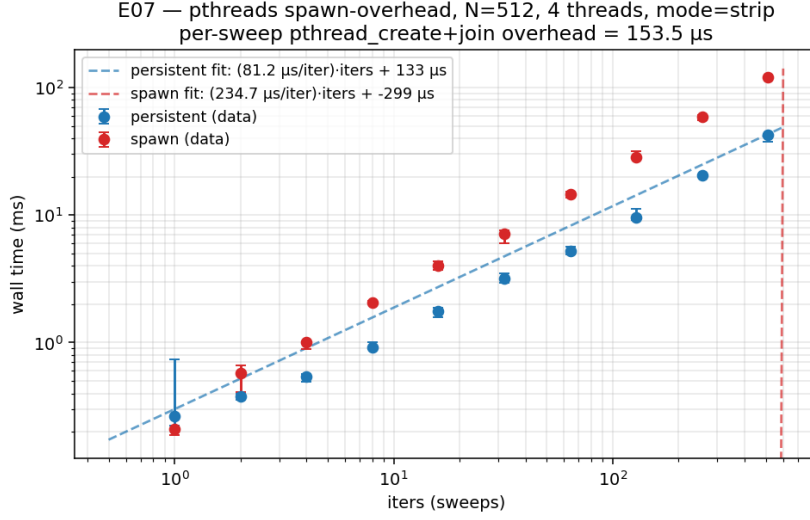


Figure 7: Linear-fit time = $a + b \cdot \text{iters}$. Persistent slope: $81 \mu\text{s}/\text{sweep}$ (compute work only). Spawn slope: $235 \mu\text{s}/\text{sweep}$. **Per-sweep pthread_create+join overhead = $153 \mu\text{s}$** – larger than the $81 \mu\text{s}$ of actual compute work. Spawning per sweep is a $1.89\times$ tax on the kernel.

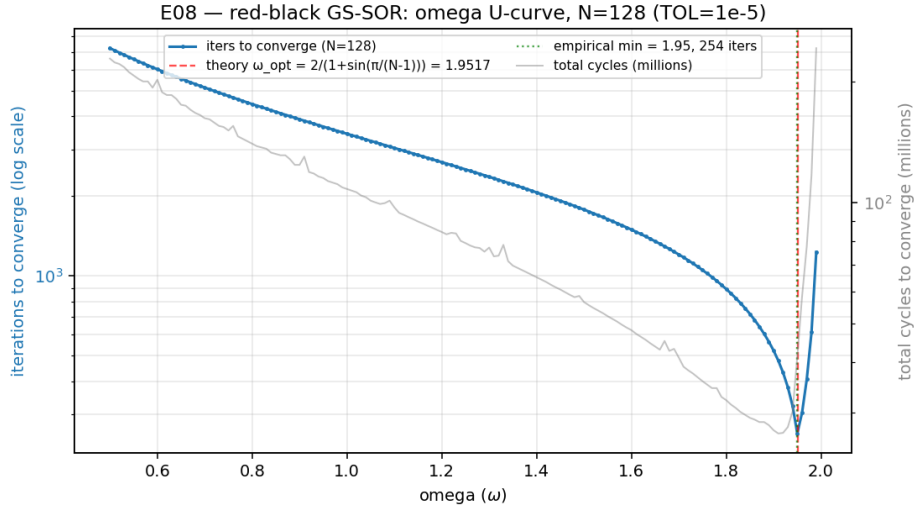
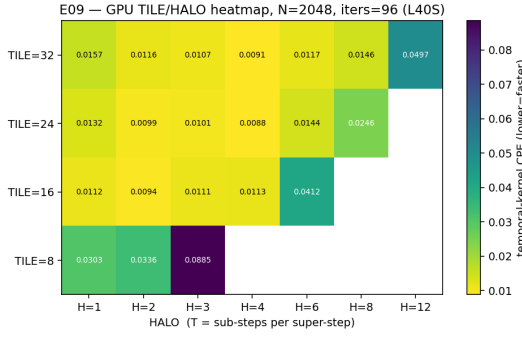
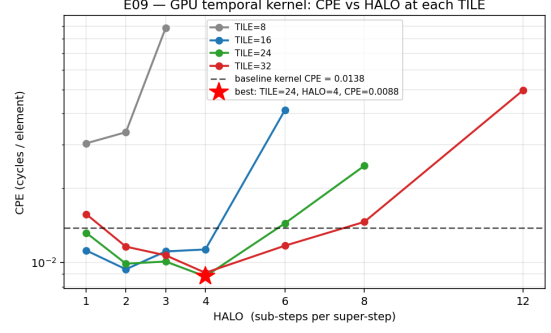


Figure 8: The classical Lab-5 figure. 150 ω values in $[0.50, 1.99]$ at $N=128$, log-y. Theoretical $\omega_{\text{opt}} = 2/(1 + \sin(\pi/(N-1))) = 1.9517$ (red dashed) matches the empirical minimum at $\omega=1.95$ (green dotted) to **0.1 %**. 254 iters at ω_{opt} vs 3,434 at $\omega=1$ (Gauss-Seidel) – a **$13.5\times$ fewer-iteration win** from algorithmic tuning alone.

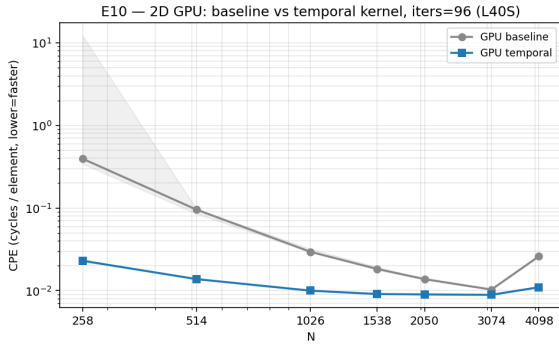


(a) Heatmap, dark = best CPE.

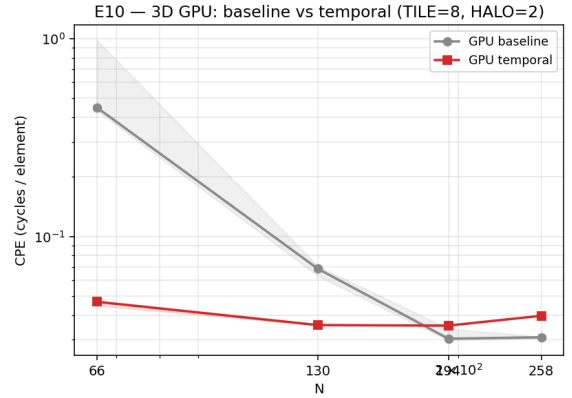


(b) CPE vs HALO, one line per TILE. ★ = global min.

Figure 9: 21 (TILE, HALO) combos at $N=2048$, iters= 96 on L40S. Best at (TILE=24, HALO=4) with CPE 0.0088 – **$1.56\times$ over the per-launch baseline kernel**. HALO=4 wins for all $TILE \geq 16$; TILE=8 is dead (only 64 threads/block). Empirical pattern: **keep HALO $\leq$ INTER/4**.



(a) 2D GPU: temporal wins 1.5–17 $\times$.



(b) 3D GPU: temporal *loses* at $N \geq 194$.

Figure 10: Both kernels validate against the CPU reference at $\sim 1.4 \times 10^{-7}$ relative error (FP32 round-off over 96 iters). Best 2D GPU CPE = **0.0089** at $N=3074$; both kernels degrade at $N=4098$ when the working set exceeds the L40S L2 (~ 96 MB) – the GPU equivalent of the CPU L3 cliff in E02. The 3D temporal regression is a known halo-overhead issue ($7\times$ redundant work at TILE=8/HALO=2); the 2.5D-streaming variant is the standard fix.

8.9 GPU TILE/HALO sweep (E09)

8.10 GPU baseline vs temporal across N (E10)

8.11 Cross-tier headline (E11)

9 Summary table of best CPE per tier

10 Conclusions and future work

The project demonstrates the full optimisation stack for a structured- grid SOR solver, with each layer’s contribution measurable in isolation and reproducible to within run-to-run noise. The headline result is the **$310\times$ cross-tier span** at $N=2050$ (Figure 11); the most informative single chart for the algorithm is the **T -curve at fixed $B=128$** (Figure 4), which makes the linear-savings vs quadratic- overhead tradeoff visible at a glance.

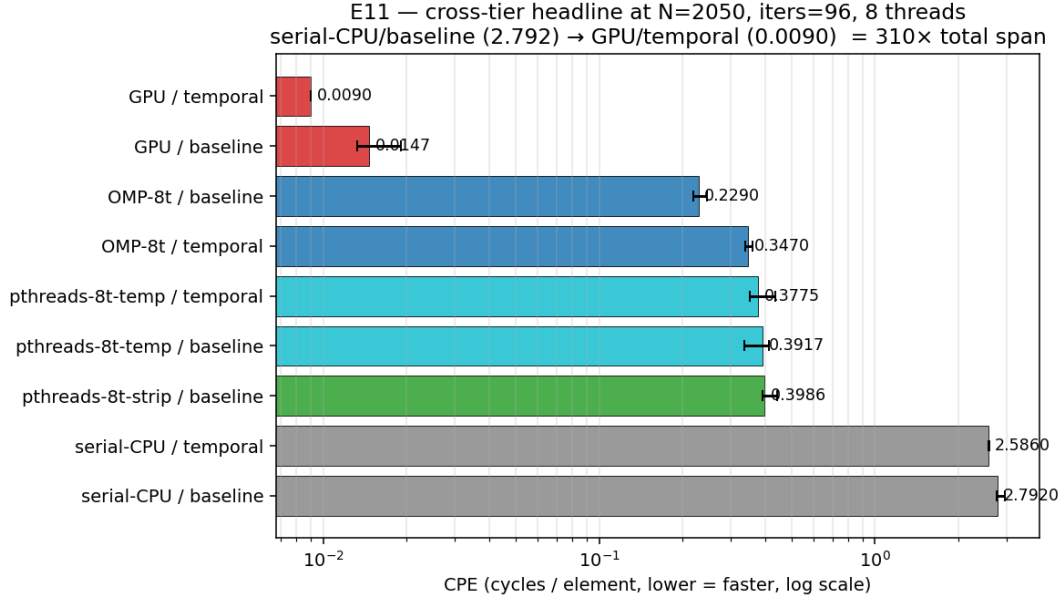


Figure 11: Every variant on one axis at $N=2050$, iters= 96, 8 threads. Slowest (serial-CPU baseline, 2.792 CPE) to fastest (GPU temporal, 0.0090 CPE) is a **310 $\times$ span**. Each step is attributable: serial $\rightarrow$ 8-cores ($\sim 10\times$) $\rightarrow$ GPU ($\sim 15\times$) $\rightarrow$ shared-memory temporal ($\sim 1.6\times$). Error bars: min/max across 3 reps.

Tier	Best CPE	Speedup vs serial	Source	Config
Serial CPU baseline	2.792	1.00 $\times$	E11	$N=2050$
Serial CPU temporal	2.586	1.08 $\times$	E11	$N=2050$
pthreads-strip persistent, 8t	0.255	10.9 $\times$	E05	$N=2048$, iters= 64
OMP-8t baseline	0.229	12.2 $\times$	E11	$N=2050$
OMP-8t temporal	0.332	8.4 $\times$	E04	$N=2048$, $T=8$
GPU baseline (2D)	0.0103	271 $\times$	E10	$N=3074$
GPU temporal (2D)	0.0088	317$\times$	E09	TILE=24, HALO=4

Table 1: Best CPE per tier across all experiments. At CPNS= 2.0, GPU temporal’s 0.0088 CPE corresponds to ~ 4.4 ps per element on the L40S – about one floating-point op per nanosecond per cell, near the Roofline machine-balance line for a memory-bound 5-point stencil at FP32.

Three concrete future-work items, in priority order:

1. **Red-black GS-SOR temporal kernel.** Combine the 13.5 $\times$ algorithmic win from E08 with the per-iter speedups of E04/E09. Requires re-deriving the shrinking trapezoid for paired half-sweeps; probably 1–2 days of careful indexing.
2. **2.5D-streaming for the 3D GPU temporal kernel** (Micikevicius 2009). Slide a 3-z-plane window through shared memory instead of loading a cubic tile; turns 3D’s 7 $\times$ halo overhead into 2D’s 0.78 $\times$.
3. **Multigrid V-cycle.** The natural way to amplify SOR’s per-iter speedup with a 5–20 $\times$ fewer-iterations win on top. V-cycle restriction/prolongation is straightforward; the per-level red-black smoother is the existing E08 kernel.

Reproducibility

All raw run logs, derived CSVs, and figures are checked in under `results/exp/E0{0--11}`. Every per-experiment directory has a `notes.md` with the headline finding and the specific parameter set. `results/STATUS.md` is the one-glance dashboard. The companion Markdown report at `report.md` shares this section's structure for easy diffing.

Build:

```
module load cuda/12.8 && make && make gpu
```

Re-run any single experiment: scripts in `experiment_execution.md` (E01–E11) reproduce the bench in ~30 minutes total.